

Git for Developers

Module 7 — Optimization & Best Practices

Commit Conventions, Branch Naming, Aliases, and Performance

Module 7 — Agenda

Writing Better Git History

- Writing good commit messages — Conventional Commits
- Atomic commits: one logical change per commit
- Branch naming conventions

Keeping Things Manageable

- Avoiding merge hell
- Git aliases and shortcuts
- Performance tips for large repositories

Part 1 — Writing Good Commit Messages

Why Commit Messages Matter

A commit message is a letter to your future self and your teammates:

- `git log` becomes meaningful documentation
- Code review is faster with clear context
- `git bisect` can pinpoint bugs from the message alone
- `git blame` tells a story, not just who typed what

```
# Bad log
```

```
$ git log --oneline
```

```
a3f7c21 update
```

```
b2e1d40 fix
```

```
c9a8b12 changes
```

```
d7f3e56 stuff
```

```
# Good log
```

```
$ git log --oneline
```

```
a3f7c21 feat: add JWT refresh token rotation
```

```
b2e1d40 fix: prevent login bypass on expired token
```

```
c9a8b12 docs: update auth API reference
```

```
d7f3e56 chore: upgrade express to 4.19.0
```

The Seven Rules of a Great Commit Message

1. **Separate subject from body** with a blank line
2. **Limit the subject** to 50 characters
3. **Capitalize the subject** line
4. **Do not end the subject** with a period
5. **Use the imperative mood** in the subject: "Fix bug" not "Fixed bug"
6. **Wrap the body** at 72 characters
7. **Explain the why** in the body, not the what

The **what** is visible in the diff. The commit message should explain the **why**.

Conventional Commits

A structured commit format adopted by many teams:

```
<type>(<scope>): <short description>
```

```
[optional body]
```

```
[optional footer]
```


Types

Type	Use for
feat	A new feature
fix	A bug fix
docs	Documentation only
chore	Build, tooling, dependencies
refactor	Code restructuring (no behavior change)
test	Adding or fixing tests
ci	CI/CD configuration changes

Conventional Commits — Examples

`feat(auth): add refresh token rotation`

Implement automatic token refresh to reduce re-login friction.

- Refresh tokens are rotated on every use (sliding window)
- Old tokens are invalidated immediately after rotation
- Expiry is configurable via `AUTH_REFRESH_EXPIRY` env var

Closes #142

`fix(api): return 404 instead of 500 when user not found`

Unhandled null check caused a 500 when looking up a deleted user.

`chore(deps): upgrade axios from 1.5.0 to 1.7.2`

Part 2 — Atomic Commits

What is an Atomic Commit?

An atomic commit contains **exactly one logical change**:

- One bug fix
- One new feature
- One refactor
- One dependency bump

Why it matters

- Easy to review — reviewer knows exactly what changed and why
- Easy to revert — `git revert` undoes one thing cleanly
- Easy to cherry-pick — can be applied to another branch safely
- `git bisect` finds bugs at the exact commit level

Atomic vs. Non-Atomic

Non-atomic (avoid)

```
commit a3f7c21  
"fix login bug, update deps, refactor user service, add tests, fix typo in README"
```

Atomic (preferred)

```
commit a3f7c21 fix: prevent login bypass on expired token  
commit b2e1d40 test: add expired token login test cases  
commit c9a8b12 refactor: extract TokenValidator to separate class  
commit d7f3e56 chore: upgrade jsonwebtoken to 9.0.2  
commit e1c2a34 docs: fix typo in authentication README
```

Smaller commits are not just better — they're essential for team collaboration.

`git add -p` — Stage Atomically

When you've made multiple unrelated changes to the same file:

```
# Interactively stage hunks (parts of files)
git add -p src/auth.js
```

```
diff --git a/src/auth.js b/src/auth.js
@@ -12,6 +12,7 @@ function validateToken(token) {
    if (!token) return false;
+   if (isExpired(token)) return false;    ← stage this? (y/n/s/q)
    return jwt.verify(token, SECRET);
}
```

Keys: `y` (stage), `n` (skip), `s` (split hunk), `q` (quit), `?` (help)

Part 3 — Branch Naming Conventions

Branch Naming Best Practices

Pattern	Example	Use for
feature/<description>	feature/user-authentication	New features
fix/<description>	fix/login-redirect-loop	Bug fixes
hotfix/<description>	hotfix/security-patch-cve-2024	Urgent production fixes
release/<version>	release/2.4.0	Release preparation
chore/<description>	chore/upgrade-node-20	Tooling and maintenance
docs/<description>	docs/update-api-reference	Documentation

Branch Naming Rules

Good branch names

feature/user-authentication

fix/payment-null-pointer

release/v2.4.0

chore/upgrade-dependencies

Bad branch names

my-branch # no context

fix # too vague

Feature/UserAuth # inconsistent casing (use lowercase-kebab-case)

fix_login # use hyphens, not underscores

john/stuff # personal names add no value

- Use **lowercase-kebab-case**
- Be **descriptive** but concise (3-5 words)
- Include **ticket numbers** if your team uses issue trackers: feature/PROJ-142-user-auth

Part 4 — Avoiding Merge Hell

What is Merge Hell?

Merge hell is when integrating branches becomes painful:

- Long-lived branches that diverge significantly from `main`
- Hundreds of conflicting lines that must be manually resolved
- Circular dependencies between feature branches
- History that is impossible to follow

How to Avoid It

Practice	How
Keep branches short-lived	Merge within days, not weeks
Rebase frequently	<code>git rebase origin/main</code> daily to stay current
Break large features down	Ship incrementally behind feature flags
Communicate	Tell teammates when you're touching shared files
Delete merged branches	Keeps the branch list clean
Don't nest branches	Avoid <code>feature/b</code> branched from <code>feature/a</code>

```
# Set up automatic branch deletion after merge (GitHub)
# Repo Settings → General → Automatically delete head branches 
```

Part 5 — Git Aliases & Shortcuts

Creating Git Aliases

Aliases let you create shortcuts for common commands:

```
# Add aliases to your global config
git config --global alias.st status
git config --global alias.co checkout
git config --global alias.br branch
git config --global alias.ci commit
git config --global alias.lg "log --oneline --graph --all --decorate"
git config --global alias.unstage "restore --staged"
git config --global alias.last "log -1 HEAD"
```

```
# Now you can use:
git st          # git status
git co main     # git checkout main
git lg          # pretty branch graph
git unstage file.js # git restore --staged file.js
```


Useful Aliases

```
# Show the last commit
git config --global alias.last "log -1 HEAD --stat"

# List all aliases
git config --global alias.aliases "config --get-regexp ^alias"

# Undo last commit (keep changes staged)
git config --global alias.undo "reset --soft HEAD~1"

# Push current branch to origin with tracking
git config --global alias.pub "push -u origin HEAD"

# Compact log with relative dates
git config --global alias.ll \
    "log --oneline --decorate --all --graph --date=relative"
```

Part 6 — Performance Tips

Performance in Large Repositories

Git can slow down with millions of files or a very deep history:

```
# Enable the filesystem monitor (daemon watches for changes)
git config --global core.fsmonitor true
git config --global core.untrackedCache true

# Enable multi-pack index for faster object lookup
git config --global core.multiPackIndex true

# Run maintenance automatically (packs objects, updates indexes)
git maintenance start
```

Shallow Clones — Faster CI/CD

```
# Clone only the most recent commit (no history)
git clone --depth 1 https://github.com/org/repo.git

# Clone with limited history
git clone --depth 50 https://github.com/org/repo.git

# Clone a single branch (no other branches)
git clone --single-branch --branch main https://github.com/org/repo.git

# Unshallow later if you need full history
git fetch --unshallow
```

Shallow clones can reduce clone time from minutes to seconds in large repos.
Ideal for CI pipelines.

Partial Clone — Only Download What You Need

```
# Clone without downloading blob objects (file contents) upfront  
git clone --filter=blob:none https://github.com/org/large-repo.git  
  
# Clone only tree objects (no blobs or commit objects)  
git clone --filter=tree:0 https://github.com/org/large-repo.git
```

Git will fetch missing objects on demand as you access them — ideal for large monorepos.

`.gitignore` and Repository Size

```
# See what's taking up space in your repo
git count-objects -vH

# Find the largest objects in your history
git rev-list --objects --all | \
  git cat-file --batch-check='%(objecttype) %(objectname) %(objectsize) %(rest)' | \
  sort -k3 -rn | head -20

# Remove a large file from history (use BFG Repo Cleaner or git-filter-repo)
git filter-repo --strip-blobs-bigger-than 10M
```

Never commit binaries, build artifacts, or secrets to Git. Use `.gitignore` proactively.

Summary — Best Practices

Topic	Key takeaway
Commit messages	Conventional Commits: <code>type(scope): description</code>
Commit size	One logical change per commit (atomic)
Branch names	<code>feature/</code> , <code>fix/</code> , <code>hotfix/</code> + lowercase-kebab-case
Merge hell	Keep branches short-lived, rebase often
Aliases	<code>git lg</code> , <code>git st</code> , <code>git undo</code> — save keystrokes
Performance	<code>--depth 1</code> for CI, <code>core.fsmonitor</code> , <code>git maintenance</code>